CAMPUS DE EXCELENCIA INTERNACIONAL

Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Metodologías de Desarrollo Web

Máster de Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026



Agustín Yagüe y Jennifer Pérez
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos

CAMPUS DE EXCELENCIA INTERNACIONAL

Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

- **CONTENIDOS**
  - **Behaviour-Driven Development (BDD)**
  - **Testing - JUnit**
  - **Test-Driven Development**



¿Qué es JUnit?

- JUnit es un Framework Open Source para la automatización de las pruebas en los proyectos software
- Se basa en las ideas de Kent Beck
- ¿Por qué probar JUnit?

Todos los programadores cometen errores, por lo que hay que intentar encontrar esos errores lo más pronto posible

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

¿Qué es JUnit?

- Las pruebas unitarias del código ayudan al desarrollador a verificar que la lógica una pieza del programa es correcta
- JUnit crea pequeñas unidades que revisen funcionalidades puntuales del código y probando que funcionen como debe
- La ejecución automática de pruebas ayuda a identificar las regresiones de software introducidas por los cambios en el código fuente.
- Contar con una alta cobertura de pruebas del código permite continuar desarrollando funciones sin tener que realizar muchas pruebas manuales.

JUnit

- Tipos de pruebas:
  - Unitarias (internas a nivel de clase o fichero Java)
  - Integración (prueba relaciones entre clases y/o tecnologías)
  - Funcionales (pruebas sobre la funcionalidad)

JUnit

- Bases:
  - Las pruebas se agrupan en clases de prueba
  - Las clases de prueba heredan de "**junit.framework.TestCase**" o utilizan '**static imports**'
  - Las clases contienen métodos de prueba que empiezan por la palabra **test*** y deben de tener un nombre significativo

- En JUnit4 se puede utilizar la anotación **@Test** para Indicar a Junit que se trata de un método de Test.
- De esta forma es posible utilizar "*naming conventions*" alternativos para nombrar los tests: "Given[ExplainYourInput]When[WhatIsDone]Then[Expected Result]"

JUnit 4

- ¿Qué es una prueba unitaria?
  - Es una porción de código que ejecuta otra porción de código en un contexto conocido, con unas determinadas entradas y que compara los resultados esperados mediante una sentencias especiales : **ASSERT**

```
public class Calculadora {

    public int sumar(int i, int j) {
      return i+j;
    }
}
```

  - Cada sentencia '**assert**' está pensada para realizar una comparación especifica

```
import static org.junit.Assert.*;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;

public class CalculadoraTest {

   Calculadora c;

   @Before
   public void setUp()  {
     c = new Calculadora();
   }

  @After
   public void tearDown() {
   }

  @Test
   public void sumar() {
     assertEquals(10, c.sumar(6, 4));
   }
```


JUnit 4

- Compartir datos entre casos de prueba (**Fixtures**)
  - Permite eliminar redundancia de código dentro de las pruebas
  - Hay varias situaciones:
    - Acciones que se tienen que hacer **antes/después de ejecutar todas las pruebas**
      - **@BeforeClass()**: se ejecuta al cargar la clase. Ideal para inicializar contextos (setUpClass)
      - **@AfterClass()**: se ejecuta al acabar de ejecutar todas las pruebas. Ideal para cerrar recursos (tearDownClass)
    - Acciones que se tienen que hacer **antes/después de ejecutar cada método de prueba**
      - **setUp():** es llamado antes de ejecutar cada test. Permite establecer el estado compartido por varios tests
      - **tearDown()**: es llamado después de ejecutar cada test. Permite cerrar recursos compartidos por varios tests

Esqueleto de una prueba unitaria

```
Import // junit libraries
class Class_Name_Test extends junit.framework.TestCase {
    @BeforeClass
    public static void setUpClass() throws Exception {
            //Inicialización general de variables, escritura del log...
    }
    @AfterClass
    public static void tearDownClass() throws Exception {
            //Liberación de recursos, escritura en el log...
    }
    @Before
      public void setUp() {
            //Inicialización de variables antes de cada Test
      }
    @After
      public void tearDown() {
            //Tareas a realizar después de cada test
      }
    @Test
    public function testName1()  {
          // inicialización del contexto
            Ejecutar método
          // comprobar aserción
    }
}
```


JUnit en detalle

- Probando Excepciones
  - Tan importante es detectar que el código está bien como que captura de manera adecuada las excepciones en bloques **try_catch**
  - Se utilizan anotaciones para probar excepciones
    @Test(expected = Nombre_de_la_Excepción)
  - En el caso de que el test lance la excepción se considera correcto

```
@Test(expected = ArithmeticException.class)
    public void dividirPorCero() {
            c.dividir(5, 0);
    }
```

```
@Test
public void expresionErronea() {
    try {
            c.operacion("( ?1w5 a 2)");
            fail("No salta excepcion");
        } catch (CalculadoraExpresionException ex) {
            assertTrue(true);
        }
}
```

Junit 4 - Assertions

Table 2. Methods to assert test results

Statement	Description
fail([message])	Let the method fail. Might be used to check that a certain part of the code is not reached or to have a failing test before the test code is implemented. The message parameter is optional.
assertTrue([message,] boolean condition)	Checks that the boolean condition is true.
assertFalse([message,] boolean condition)	Checks that the boolean condition is false.
assertEquals([message,] expected, actual)	Tests that two values are the same. Note: for arrays the reference is checked not the content of the arrays.
assertEquals([message,] expected, actual, tolerance)	Test that float or double values match. The tolerance is the number of decimals which must be the same.
assertNull([message,] object)	Checks that the object is null.
assertNotNull([message,] object)	Checks that the object is not null.
assertSame([message,] expected, actual)	Checks that both variables refer to the same object.
assertNotSame([message,] expected, actual)	Checks that both variables refer to different objects

https://junit.org/junit4/

https://www.baeldung.com/junit-assertions

Buenas prácticas

- Usar las aserciones más adecuadas y específicas posibles
  - Utiliza aserciones simples
  - Usar el método adecuado al objetivo de la prueba

assertCount(1, array('foo')); vs. *assertEquals(1, count(array('foo')));*

assertInstanceOf('Foo', $foo);* vs. *assertTrue($foo instanceof Foo);

JUnit en detalle

- **Parameters**
  - En ocasiones se quiere realizar la misma prueba con varios conjuntos de datos
  - Los 'parameters' pueblan de datos los tests
  - Se identifican mediante anotaciones

    @Parameter **@Parameters**
  - Cada tupla de datos representa un caso de prueba

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

JUnit en detalle

```
@RunWith(Parameterized.class)
public class CalculadoraTest {

  Calculadora c;

  @Parameter(0)
   public int op1;
   @Parameter(1)
   public int op2;
   @Parameter(2)
   public int result;

@Parameters
   public static Object[][] data() {
      return new Object[][] {
                  { 2, 2, 4 },
                  { 2, 3, 5 },
                  { 2, 4, 6 },
      };
   }
```

```
@Test
  public void sumar() {
            assertEquals(result, c.sumar(op1, op2));
            }
}
```

MDW: Metodologías Web

Buenas prácticas

- Desvincular la prueba de los datos
  - Los test pueden aceptar argumentos, estos argumentos deben ser proporcionados por un método proveedor
  - Etiquetar a los métodos proveedores con la notación ***@dataProvider***

```
@RunWith(DataProviderRunner.class)
public class DataProviderArraySyntaxTest {

   @DataProvider
   public static Object[][] dataProviderAdd() {
      // @formatter:off
      return new Object[][] {
               {0, 0, 0},
               {1, 1, 2},
               /* ... */
      };
      // @formatter:on
   }

   @Test
   @UseDataProvider("dataProviderAdd")
   public void testAdd(int a, int b, int expected) {
      // When:
      int result = a + b;
      // Then:
      assertEquals(expected, result);
   }
}
```


Buenas prácticas

- Propiedades de un buen test:
  - Independiente
  - Rápido
  - Repetible
  - Actualizado
  - Cortos
  - Si no cambia el código no debe de variar

Buenas prácticas

- Separar el código de la aplicación de las pruebas
- Normalmente, las pruebas unitarias se crean en un proyecto independiente o en una carpeta de código fuente independiente para mantener el código de prueba separado del código real.
- La convención estándar de las herramientas de compilación Maven y Gradle es utilizar :

src/main/java - for Java classes
src/test/java - for test classes

Maven

```
<dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.12</version>
        <scope>test</scope>
</dependency>
```


CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Buenas prácticas

Más buenas prácticas:

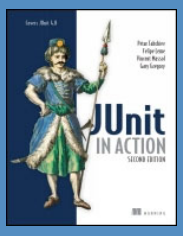
- Sigue los pasos de TDD, comprueba que el método falla justo después de crearlo.
- Refactoriza a menudo, sino el código duplicado no te dejará dormir por las noches.
- Minimiza el número de assert en cada test.
- Todas las pruebas deben funcionar antes de escribir la siguiente.
- No introducir dependencias entre pruebas, la frase “como el test XXX viene antes podemos probar YYY así” no es buena.

MDW: Metodologías Web

References

References

- https://www.ibm.com/developerworks/java/tutorials/j-junit4/
- JUnit in Action, Second Edition



Metodologías de Desarrollo Web

Máster de Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026



Agustín Yagüe y Jennifer Pérez
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos